

ROADMAP MASTER CUB3D — GUIA DEFINITIVO DE CONSTRUÇÃO E ESTUDO

Documento técnico e pedagógico estruturado para o desenvolvimento passo a passo do projeto **cub3D** (42 Common Core).

Abrange desde os fundamentos matemáticos e gráficos da miniLibX até o Raycasting com DDA, arquitetura em C, fluxo Git profissional por branches, implementação de todos os bônus, pipeline de assets retrô e cronograma detalhado.

ÍNDICE

1. Visão Geral, Filosofia e Aprendizados Essenciais
2. Fundamentos Teóricos e Matemáticos de Computação Gráfica
3. Guia Aprofundado da miniLibX (MLX) e Framebuffer
4. Arquitetura de Software e Estruturas de Dados (`include/`)
5. Roadmap Passo a Passo por Fases e Git Workflow
 - Fase 0: Setup, Makefile, Libft e Esqueleto
 - Fase 1: Parsing Rigoroso e Validação do Mapa (.cub)
 - Fase 2: Motor Gráfico MLX, Framebuffer e Céu/Chão
 - Fase 3: O Motor de Raycasting (DDA e Paredes Sólidas)
 - Fase 4: Movimento Suave, Câmera e Colisão Básica
 - Fase 5: Mapeamento de Texturas (XPM) nas Paredes
 - Fase 6: Auditoria da Parte Obrigatória (Norma, Leaks, Edge Cases)
 - Fase 7 (Bônus 1): Sistema Avançado de Colisão e Deslize
 - Fase 8 (Bônus 2): Minimapa 2D Interativo no HUD
 - Fase 9 (Bônus 3): Portas Interativas (Abertura/Fechamento)
 - Fase 10 (Bônus 4): Rotação de Visão com o Rato (Mouse Look)
 - Fase 11 (Bônus 5): Sprites Animadas e Z-Buffer (Depth Buffer)
 - Fase 12: Integração Final, README Oficial e Preparação para Defesa
6. Manual de Assets: Criação, Obtenção e Conversão para XPM
7. Estimativa de Prazos e Cronograma a partir de Hoje
8. Erros Clássicos a Evitar e Dicas para a Avaliação por Pares

1. Visão Geral, Filosofia e Aprendizados Essenciais

1.1 O que é o cub3D?

O projeto **cub3D** da Escola 42 é uma homenagem ao clássico **Wolfenstein 3D** (desenvolvido em 1992 por John Carmack e John Romero na id Software). Ele marca a transição da programação de sistemas e manipulação de texto em C para o fascinante universo da **computação gráfica de baixo nível**.

1.2 Por que Raycasting e não Rasterização 3D ou Raytracing?

Para entender a genialidade do cub3D, compare as três principais abordagens gráficas:

Paradigma	Como funciona	Complexidade Computacional	Aplicação Histórica / Atual
Rasterização 3D	Projeta polígonos (triângulos) 3D na tela 2D usando matrizes MVP (Model-View-Projection), shaders e GPUs.	Moderada (acelerada por hardware moderno).	OpenGL, DirectX, Vulkan, Quake (1996) em diante.
Raytracing Clássico	Simula a física real da luz: dispara raios por cada pixel, calculando reflexões, refrações e sombras secundárias recursivamente.	Altíssima ($O(N \times \text{raios recursivos})$).	Cinema, Renderizadores offline (Blender Cycles), RTX em GPUs modernas.
Raycasting (2.5D)	O mundo é um grid 2D visto de cima. Dispara-se apenas um raio por coluna horizontal da tela (ex: 1200 raios para uma tela de 1200x800). As paredes têm altura uniforme e ângulos de 90°.	Mínima (roda a centenas de frames por segundo em CPU pura sem GPU).	Wolfenstein 3D (1992), cub3D.

💡 **Conceito Chave:** O mundo do cub3D não é verdadeiramente 3D em termos de polígonos. É uma simulação 2D vista sob perspectiva cilíndrica vertical. Cada coluna vertical da tela corresponde a um único raio disparado no plano 2D.

1.3 O que o projeto vai te ensinar na prática?

- 1. **Manipulação de Memória Direta em Gráficos:** Como transformar uma matriz ou cálculo geométrico em pixels de cor escritos linearmente na memória RAM através de um *framebuffer*.
- 2. **Álgebra Linear e Geometria Vetorial Aplicadas:** Vetores unitários, matrizes de rotação 2D, trigonometria analítica e o plano de projeção de câmera.
- 3. **Algoritmos Eficientes:** O algoritmo **DDA (Digital Differential Analysis)** para travessia de grades discretas em tempo linear, e o algoritmo de **Flood Fill** para validação topológica de mapas.
- 4. **Resiliência e Arquitetura de Software em C:** Parsing robusto capaz de resistir a entradas corrompidas, gerenciamento rigoroso de memória sem vazamentos (*zero leaks* no Valgrind) e modularidade estrita respeitando a Norma 42.
- 5. **Engenharia de Jogos Clássica:** O *Game Loop* desacoplado, mapeamento de texturas sem distorção e o *Z-Buffering* para ordenação de profundidade de objetos e sprites.

2. Fundamentos Teóricos e Matemáticos de Computação Gráfica

2.1 Sistemas de Coordenadas: Mundo vs. Tela

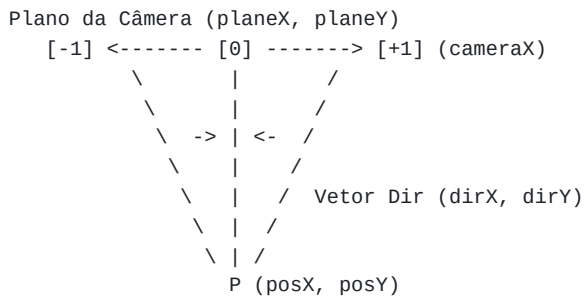
No motor do cub3D, convivem dois espaços de coordenadas:

- 1. **Coordenadas do Mundo (World Space - Ponto Flutuante):**
 - A posição do jogador é um par ordenado real: $(pos_x, pos_y) \in \mathbb{R}^2$ (ex: $x = 3.5, y = 8.2$).
 - O mapa é uma grade discreta de blocos inteiros: $(map_x, map_y) \in \mathbb{Z}^2$.
- 2. **Coordenadas da Tela (Screen Space - Inteiros):**
 - A janela possui dimensões fixas (ex: $W = 1200, H = 800$).
 - O eixo X da tela vai de 0 a $W - 1$ (cada coluna recebe um raio).
 - O eixo Y da tela vai de 0 (topo) a $H - 1$ (base).

2.2 Vetores de Direção e o Plano da Câmera (Evitando Funções Trigonômicas Lentas)

Muitos iniciantes tentam calcular a visão do jogador usando ângulos em radianos (θ) e chamando $\cos(\theta)$ e $\sin(\theta)$ para cada raio. Isso é lento e gera complexidade desnecessária.

A técnica de John Carmack (popularizada pelo tutorial clássico de Lode Vandevenne) baseia-se puramente em **vetores 2D**:



- Vetor de Posição:** $\vec{pos} = (pos_x, pos_y)$
- Vetor de Direção:** $\vec{dir} = (dir_x, dir_y)$, com $|\vec{dir}| = 1$ (vetor unitário).
- Vetor do Plano da Câmera:** $\vec{plane} = (plane_x, plane_y)$, perpendicular a $\vec{dir}$.
 - O comprimento de $\vec{plane}$ define o **Campo de Visão (FOV - Field of View)**.
 - Se $|\vec{dir}| = 1.0$ e $|\vec{plane}| = 0.66$, o campo de visão é: $\text{FOV} = 2 \times \arctan(0.66 / 1.0) \approx 66^\circ$ (o campo de visão padrão de Wolfenstein 3D e Doom).

Para cada coluna x da tela (de 0 a $W - 1$):

- Mapeia-se x para o espaço de câmera normalizado: $\text{cameraX} = 2 \times \frac{x}{W} - 1 \in [-1.0, 1.0]$
- O vetor de direção do raio daquela coluna é uma simples soma vetorial: $\vec{rayDir} = \vec{dir} + \vec{plane} \times \text{cameraX}$
 $rayDir_x = dir_x + plane_x \times \text{cameraX}$
 $rayDir_y = dir_y + plane_y \times \text{cameraX}$

2.3 O Algoritmo DDA (Digital Differential Analysis)

Se tentássemos avançar o raio de 0.01 em 0.01 no espaço contínuo, gastaríamos milhares de iterações e ainda poderíamos "atravessar" a quina de uma parede por aproximação.

O **DDA** resolve isso de forma exata: ele salta diretamente de linha em linha da grade do mapa (seja uma linha vertical x ou horizontal y).

As Variáveis do DDA:

- $mapX, mapY$: A célula inteira do grid onde o raio está no momento ($(int)posX, (int)posY$).
- $deltaDistX, deltaDistY$: A distância que o raio deve percorrer para cruzar 1 bloco completo na direção X ou Y.
 $\text{deltaDistX} = \left\lceil \frac{1}{rayDir_x} \right\rceil$ $\text{deltaDistY} = \left\lceil \frac{1}{rayDir_y} \right\rceil$
(Dica: se $rayDir_x == 0$, $\text{deltaDistX} = 1e30$, um valor astronômico para evitar divisão por zero).
- $stepX, stepY$: Sentido do passo (+1 se $rayDir > 0$, -1 se $rayDir < 0$).
- $sideDistX, sideDistY$: A distância da posição inicial do jogador até a PRIMEIRA borda do bloco no eixo X e Y.

```
// Inicialização do DDA
if (rayDirX < 0) {
    stepX = -1;
    sideDistX = (posX - mapX) * deltaDistX;
} else {
    stepX = 1;
    sideDistX = (mapX + 1.0 - posX) * deltaDistX;
}
if (rayDirY < 0) {
    stepY = -1;
    sideDistY = (posY - mapY) * deltaDistY;
} else {
    stepY = 1;
    sideDistY = (mapY + 1.0 - posY) * deltaDistY;
}
```

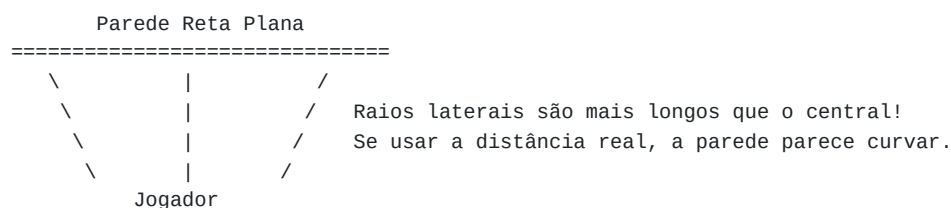
O Loop de Execução:

Em cada iteração do loop:

- Se `sideDistX < sideDistY`: avança no eixo X (`sideDistX += deltaDistX`; `mapX += stepX`; `side = 0`).
- Caso contrário: avança no eixo Y (`sideDistY += deltaDistY`; `mapY += stepY`; `side = 1`).
- Se `map[mapY][mapX] == '1'` : **PARE!** O raio encontrou uma parede. A variável `side` indica se foi uma face vertical (`side = 0`) ou horizontal (`side = 1`).

2.4 O Efeito Fish-Eye e a Distância Perpendicular

Se medirmos a distância euclidiana direta do jogador até o impacto da parede: $d_{\text{euclidiana}} = \sqrt{(\text{impacto}_x - \text{pos}_x)^2 + (\text{impacto}_y - \text{pos}_y)^2}$ Ocorre o infame **Efeito Olho de Peixe (Fish-Eye)**: paredes perfeitamente planas parecem convexas/arredondadas, porque os raios das pontas da tela percorrem um caminho mais longo até a parede do que o raio central.



A Solução Matemática:

Projetamos a distância perpendicularmente ao plano da câmera. A fórmula derivada pelo DDA elimina o fisheye de modo imediato:

- Se bateu em face vertical (`side == 0`):

$$\text{perpWallDist} = \text{sideDistX} - \text{deltaDistX} = \frac{\text{mapX} - \text{posX} + (1 - \text{stepX}) / 2}{\text{rayDir}_x}$$
- Se bateu em face horizontal (`side == 1`):

$$\text{perpWallDist} = \text{sideDistY} - \text{deltaDistY} = \frac{\text{mapY} - \text{posY} + (1 - \text{stepY}) / 2}{\text{rayDir}_y}$$

2.5 Altura da Linha e Projeção na Tela

Com a distância perpendicular em mãos, calculamos a altura projetada da fatia da parede: $\text{lineHeight} = \left(\frac{H}{\text{perpWallDist}} \right)$ Onde H é a altura da janela em pixels (ex: 800).

Os limites verticais de desenho na tela são centralizados: $\text{drawStart} = -\frac{\text{lineHeight}}{2} + \frac{H}{2}$
 $\text{drawEnd} = \frac{\text{lineHeight}}{2} + \frac{H}{2}$ Se $\text{drawStart} < 0$, ajusta para 0 . Se $\text{drawEnd} > H$, ajusta para $H - 1$.

2.6 Mapeamento de Texturas (UV Texture Mapping)

Para aplicar uma textura (ex: imagem de 64x64 pixels) na fatia da parede:

1. Descobrir a Face Atingida:

- $\text{side} == 0$ e $\text{rayDirX} > 0$: Parede **OESTE** (WE)
- $\text{side} == 0$ e $\text{rayDirX} < 0$: Parede **LESTE** (EA)
- $\text{side} == 1$ e $\text{rayDirY} > 0$: Parede **NORTE** (NO)
- $\text{side} == 1$ e $\text{rayDirY} < 0$: Parede **SUL** (SO)

2. Calcular o Ponto de Impacto na Parede (wallX):

- Se $\text{side} == 0$: $\text{wallX} = \text{posY} + \text{perpWallDist} \times \text{rayDir}_y$
- Se $\text{side} == 1$: $\text{wallX} = \text{posX} + \text{perpWallDist} \times \text{rayDir}_x$
- Mantemos apenas a parte fracionária: $\text{wallX} = \text{wallX} - \lfloor \text{wallX} \rfloor$ (varia entre 0.0 e 1.0).

3. Calcular a Coluna da Textura (texX):

$\text{texX} = \left(\text{int} \right) (\text{wallX} \times \text{texWidth})$ (*Inverte-se $\text{texX} = \text{texWidth} - \text{texX} - 1$ para as faces adequadas para evitar que a textura fique espelhada*).

4. Interpolarmos Verticalmente (texY):

Para cada pixel y entre drawStart e drawEnd : $\text{step} = \frac{\text{texHeight}}{\text{lineHeight}}$
 $\text{texY} = (y - \text{drawStart}) \times \text{step}$ Lemos a cor do pixel em $\text{textura}[\text{texY} \times \text{texwidth} + \text{texX}]$ e pintamos no buffer da tela.

2.7 Rotação 2D com Matriz de Rotação

Quando o jogador pressiona a seta esquerda ou direita (ou move o rato), rotacionamos os vetores $\vec{\text{dir}}$ e $\vec{\text{plane}}$ pelo ângulo α usando a matriz de rotação clássica: $\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$ No código C:

```
double old_dir_x = player->dir_x;
player->dir_x = player->dir_x * cos(rot_speed) - player->dir_y * sin(rot_speed);
player->dir_y = old_dir_x * sin(rot_speed) + player->dir_y * cos(rot_speed);

double old_plane_x = player->plane_x;
player->plane_x = player->plane_x * cos(rot_speed) - player->plane_y * sin(rot_speed);
player->plane_y = old_plane_x * sin(rot_speed) + player->plane_y * cos(rot_speed);
```

3. Guia Aprofundado da miniLibX (MLX) e Framebuffer

3.1 O Que é a MiniLibX?

A miniLibX é uma biblioteca gráfica didática desenvolvida para os estudantes da 42, atuando como um invólucro (*wrapper*) simplificado sobre o **X-Window System (X11 / Xlib)** no Linux e sobre a biblioteca **Cocoa** no macOS.

3.2 O Pecado Mortal: `mlx_pixel_put` vs. O Framebuffer em Memória

No arquivo inicial de teste `src/main.c`, existe a chamada:

```
mlx_pixel_put(mlx_ptr, win_ptr, 100, 100, 0xFF0000);
```

⚠️ AVISO CRÍTICO: NUNCA utilize `mlx_pixel_put` dentro do loop de renderização do cub3D!

No X11, cada `mlx_pixel_put` envia uma mensagem de protocolo de rede via IPC ao servidor gráfico para pintar um único pixel. Para uma tela de 1200x800 (\$960.000\$ pixels), o jogo rodará a **1 ou 2 frames por segundo**.

A Solução Profissional: O Buffer de Imagem (Off-screen Framebuffer)

Criamos uma única imagem na memória RAM com `mlx_new_image`. Desenhamos todos os 960.000 pixels diretamente na memória local e, ao final do frame, enviamos a imagem pronta para a janela de uma só vez com `mlx_put_image_to_window`.

```
[Memória RAM Local: Array de 1200 x 800 x 4 bytes]
(Raycasting escreve diretamente aqui em nanosegundos)
    |
    ▼ (1 única chamada por frame)
  mlx_put_image_to_window()
    |
    ▼
[Janela Exibida na Tela - 60+ FPS]
```

Como calcular o endereço de memória do pixel:

```
typedef struct s_img {
    void    *img_ptr;
    char    *data_addr;
    int     bpp;           // Bits per pixel (normalmente 32)
    int     line_len;      // Tamanho em bytes de uma linha horizontal
    int     endian;        // 0 para Little Endian, 1 para Big Endian
} t_img;

void    my_pixel_put(t_img *img, int x, int y, int color)
{
    char    *dst;

    if (x < 0 || x >= WIN_WIDTH || y < 0 || y >= WIN_HEIGHT)
        return ;
    dst = img->data_addr + (y * img->line_len + x * (img->bpp / 8));
    *(unsigned int *)dst = color;
}
```

3.3 Codificação de Cores em Hexadecimal (ARGB)

As cores são representadas como números inteiros de 32 bits no formato `0x00RRGGBB`:

- Byte 3 (mais significativo): Transparência / Alpha (`0x00` opaco)
- Byte 2: Canal Vermelho (Red: `0` a `255`)
- Byte 1: Canal Verde (Green: `0` a `255`)
- Byte 0 (menos significativo): Canal Azul (Blue: `0` a `255`)

Conversão das cores parseadas do .cub (F e C) para int:

```
int create_trgb(int t, int r, int g, int b)
{
    return (t << 24 | r << 16 | g << 8 | b);
}
```

Shading / Sombra Direcional:

Para conferir sensação tridimensional imediata de iluminação antes mesmo de aplicar texturas, escurecemos as faces horizontais (`side == 1`):

```
int apply_shadow(int color, double factor)
{
    int r = ((color >> 16) & 0xFF) * factor;
    int g = ((color >> 8) & 0xFF) * factor;
    int b = (color & 0xFF) * factor;
    return (r << 16 | g << 8 | b);
}
```

3.4 O Sistema de Eventos e a Técnica Anti-Stuttering

Por padrão, se você capturar teclas apenas com `mlx_key_hook`, o sistema operacional impõe um *delay de repetição de tecla* (o jogador dá um passo, congela por 500ms, e só então começa a andar continuamente aos solavancos).

A Técnica do Array de Teclas (Input Polling):

1. No `t_game`, crie uma estrutura `t_keys` com booleanos: `w`, `a`, `s`, `d`, `left`, `right`, `esc`.
2. Conecte dois hooks:
 - `mlx_hook(win, 2, 1L << 0, key_press_handler, &game); (KeyPress)`
 - `mlx_hook(win, 3, 1L << 1, key_release_handler, &game); (KeyRelease)`
3. Ao pressionar, marque a flag correspondente como `true`. Ao soltar, marque como `false`.
4. Conecte o loop de renderização contínuo:
 - `mlx_loop_hook(mlx, game_loop, &game);`
5. A cada frame do `game_loop`:
 - Atualiza o movimento se as teclas estiverem ativas.
 - Renderiza o frame completo no buffer.
 - Chama `mlx_put_image_to_window`.

Fechamento Limpo (Sem Leaks e sem Travamento):

Capture tanto a tecla ESC (`keycode 65307` no Linux) quanto o clique no botão "X" vermelho da janela (Evento 17 do X11: `DestroyNotify`):

```
mlx_hook(game.win, 17, 0, close_game, &game);
```

4. Arquitetura de Software e Estruturas de Dados (`include/`)

A divisão do código deve manter uma separação cirúrgica entre as responsabilidades. Abaixo está a arquitetura canônica recomendada:

```
include/  
├─ cub3d.h           # Protótipos de funções, includes de sistema e defines  
├─ cub3d_structs.h   # Definição de todas as structs do jogo  
└─ cub3d_bonus.h     # Extensões específicas dos bônus (minimap, sprites, portas)
```

4.1 Estruturas de Dados Essenciais (cub3d_structs.h)

```
#ifndef CUB3D_STRUCTS_H
# define CUB3D_STRUCTS_H

# include <stdbool.h>

// Buffer de imagem genérico da MiniLibX
typedef struct s_img {
    void    *img_ptr;
    char    *data_addr;
    int     bpp;
    int     line_len;
    int     endian;
    int     width;
    int     height;
}    t_img;

// Dados de uma textura carregada
typedef struct s_texture {
    char    *path;
    t_img   img;
}    t_texture;

// Estado das teclas pressionadas (para movimento suave)
typedef struct s_keys {
    bool    w;
    bool    a;
    bool    s;
    bool    d;
    bool    rot_left;
    bool    rot_right;
    bool    door_interact; // Bónus
}    t_keys;

// Jogador: posição no mundo e vetores de visão
typedef struct s_player {
    double  pos_x;
    double  pos_y;
    double  dir_x;
    double  dir_y;
    double  plane_x;
    double  plane_y;
    double  move_speed;
    double  rot_speed;
}    t_player;

// Informações do mapa e da cena (.cub)
typedef struct s_map {
    char    **grid;
    int     width;
    int     height;
    char    *no_path;
    char    *so_path;
    char    *we_path;
    char    *ea_path;
    int     floor_color;
    int     ceiling_color;
    int     player_count;
    char    player_dir;
    int     player_spawn_x;
    int     player_spawn_y;
}    t_map;

// Variáveis calculadas por coluna de raio
typedef struct s_ray {
    double  camera_x;
    double  ray_dir_x;
    double  ray_dir_y;
```

```

    int    map_x;
    int    map_y;
    double side_dist_x;
    double side_dist_y;
    double delta_dist_x;
    double delta_dist_y;
    double perp_wall_dist;
    int    step_x;
    int    step_y;
    int    hit;
    int    side;          // 0 para vertical (X), 1 para horizontal (Y)
    int    line_height;
    int    draw_start;
    int    draw_end;
    double wall_x;
    int    tex_x;
}    t_ray;

// Struct Master: Encapsula todo o estado da aplicação
typedef struct s_game {
    void      *mlx_ptr;
    void      *win_ptr;
    t_img      frame;
    t_map      map;
    t_player   player;
    t_keys     keys;
    t_texture  textures[4]; // 0: NO, 1: SO, 2: WE, 3: EA
    double     *z_buffer;   // Largura da janela (para bônus de sprites)
}    t_game;

#endif

```

5. Roadmap Passo a Passo por Fases e Git Workflow

O Fluxo Git Profissional (Git Feature-Branch Flow)

Para garantir que o repositório permaneça sempre estável, com histórico limpo e sem quebras na branch principal, o desenvolvimento adotará o seguinte protocolo:

- 1. Branch de Integração:** develop (OU main).
- 2. Branches de Trabalho:** feature/phase-X-nome .
- 3. Regra de Ouro:** NUNCA commitar diretamente na main . Todo código em uma feature branch deve ser testado, auditado contra leaks com Valgrind e compilado sem avisos (-Wall -Wextra -Werror) antes de ser mergeado.
- 4. Ciclo Padrão de Comandos:**

```

# 1. Atualizar e criar a branch de trabalho
git checkout develop
git pull origin develop
git checkout -b feature/phase-X-nome

# 2. Desenvolver, commitar com mensagens semânticas
git add .
git commit -m "feat(module): descrição clara do recurso"

# 3. Testar localmente com rigor
# 4. Fazer merge e push
git checkout develop
git merge --no-ff feature/phase-X-nome
git push origin develop

```

Fase 0: Setup, Makefile, Libft e Esqueleto

- **Branch:** feature/phase-0-setup
- **Conceitos a Estudar:**
 - Compilação modular em C, dependências de Makefiles aninhados (libft e minilibx).
 - Flags de compilação rigorosas (-Wall -Wextra -Werror).
 - Estrutura limpa de diretórios de projeto 42.
- **O que Desenvolver:**
 - Integrar a sua libft (com get_next_line e ft_split) no diretório libft/ .
 - Ajustar o Makefile principal:
 - Regras obrigatórias: all , \$(NAME) , clean , fclean , re , bonus .
 - Compilação recursiva das bibliotecas (libft.a e libmlx.a).
 - Linkagem correta das flags de sistema no Linux: -Lminilibx -lmx -Llibft -lft -lXext -lX11 -lm .
 - Crear os headers em include/ com as structs partilhadas e proteção de inclusão múltipla (#ifndef).
 - Crear função utilitária global de saída com erro: error_exit(const char *msg, t_game *game) .
- **Como Testar:**

```
make re
./cub3d          # Deve indicar uso correto ou erro amigável sem crash
make clean && make fclean && make
# Verificar que não há relink (rodar make duas vezes seguidas: segunda deve dizer 'nothing to be done')
```

- **Git Workflow da Fase:**

```
git checkout -b feature/phase-0-setup
git add Makefile include/ libft/ src/
git commit -m "setup: inicializacao do makefile, headers e libs"
git checkout develop
git merge --no-ff feature/phase-0-setup
git push origin develop
```

Fase 1: Parsing Rigoroso e Validação do Mapa (.cub)

- **Branch:** feature/phase-1-parsing
- **Conceitos a Estudar:**
 - Análise léxica e sintática de arquivos de configuração.
 - Algoritmo de **Flood Fill** para validação de labirintos e garantia de bordas fechadas.
 - Tratamento de matrizes esparsas ou de comprimentos de linha irregulares contendo espaços.
- **O que Desenvolver:**
 - Validar a extensão do arquivo passado no primeiro argumento (argv[1] deve terminar com .cub).
 - Leitura linha a linha com get_next_line .
 - Extração dos identificadores de textura: NO , SO , WE , EA .
 - Validar que aparecem exatamente uma vez cada um.
 - Validar que os arquivos de textura existem no disco usando open(path, O_RDONLY) .
 - Extração das cores de Chão (F) e Teto (c):
 - Formato: R,G,B .
 - Validar que cada canal é um número inteiro válido no intervalo [0, 255].
 - Rejeitar formatos inválidos (vírgulas a mais, letras, espaços no meio do número, valores negativos).
 - Extração da grade do mapa:
 - O mapa é sempre o último elemento do arquivo.
 - Aceitar apenas os caracteres: '0' , '1' , 'N' , 'S' , 'E' , 'W' , e espaço ' ' .

- Converter o mapa para uma matriz regular (preenchendo linhas menores com espaços para formar um retângulo uniforme).

vi. Validação do jogador:

- Deve haver **exatamente um** ponto de spawn (N , S , E ou W).
- Guardar a posição inicial e o ângulo correspondente (ex: N $\rightarrow$ vetor de direção apontando para cima (0, -1)).

vii. **Validação de Fechamento (Flood Fill):**

- O labirinto deve ser cercado por paredes '1' .
- Nenhum espaço vazio '0' ou de spawn pode estar em contato direto com um espaço ' ' ou com a borda externa da matriz.

viii. Em qualquer falha: imprimir `Error\n<mensagem descritiva>` , liberar toda a memória alocada e sair com código de erro diferente de zero.

• Como Testar:

```
# Criar suite de testes em maps/valid e maps/invalid
./cub3d maps/invalid/no_map.cub          # Deve printar Error\n...
./cub3d maps/invalid/open_wall.cub       # Deve detectar mapa aberto
./cub3d maps/invalid/two_players.cub     # Deve rejeitar 2 jogadores
./cub3d maps/invalid/invalid_color.cub   # Deve rejeitar cores fora de [0,255]
valgrind --leak-check=full ./cub3d maps/invalid/open_wall.cub
# Zero leaks tolerados no fluxo de erro!
```

• Git Workflow da Fase:

```
git checkout -b feature/phase-1-parsing
git add src/parsing/ maps/
git commit -m "feat(parsing): parser completo de texturas, cores e flood fill de mapa"
git checkout develop
git merge --no-ff feature/phase-1-parsing
git push origin develop
```

Fase 2: Motor Gráfico MLX, Framebuffer e Céu/Chão

• Branch: feature/phase-2-mlx-engine

• Conceitos a Estudar:

- Inicialização da conexão gráfica X11 (`mlx_init` , `mlx_new_window`).
- Alocação e acesso direto à memória do buffer de pixels (`mlx_new_image` , `mlx_get_data_addr`).
- Gerenciamento de eventos de fechamento limpo da janela.

• O que Desenvolver:

- Função `init_mlx()` : cria a conexão MLX e abre a janela com dimensões definidas (`WIN_WIDTH 1200` , `WIN_HEIGHT 800`).
- Criação do framebuffer principal (`t_img frame`).
- Implementação da rotina segura `my_pixel_put(t_img *img, int x, int y, int color)` .
- Função `render_ceiling_floor(t_game *game)` :
 - Pinta a metade superior da tela (linhas y de 0 a $H/2 - 1$) com a cor do Teto (`c`).
 - Pinta a metade inferior da tela (linhas y de $H/2$ a $H - 1$) com a cor do Chão (`f`).
- Configuração dos hooks de encerramento:
 - Tecla ESC fecha a janela e chama função de destruição total.
 - Botão "X" da janela encerra o programa sem vazamentos.

• Como Testar:

```
./cub3d maps/valid/simple.cub
# A janela deve abrir suavemente com a metade superior da cor do teto e a metade inferior da cor do chão.
# Pressionar ESC deve fechar imediatamente sem falhas de segmentação.
```

- **Git Workflow da Fase:**

```
git checkout -b feature/phase-2-mlx-engine
git add src/render/ src/events/
git commit -m "feat(render): criacao do framebuffer e renderizacao de ceu e chao"
git checkout develop
git merge --no-ff feature/phase-2-mlx-engine
git push origin develop
```

Fase 3: O Motor de Raycasting (DDA e Paredes Sólidas)

- **Branch:** feature/phase-3-raycasting-dda

- **Conceitos a Estudar:**

- O algoritmo DDA implementado na prática.
- O cálculo da distância perpendicular para correção do efeito *fish-eye*.
- Renderização por fatias verticais (*column-based rendering*).

- **O que Desenvolver:**

- Loop principal de raycasting iterando a coluna x de 0 até $WIN_WIDTH - 1$.
- Cálculo de `cameraX`, `rayDirX`, `rayDirY`.
- Configuração de passos e distâncias: `deltaDistX`, `deltaDistY`, `stepX`, `stepY`, `sideDistX`, `sideDistY`.
- Execução do DDA até encontrar uma parede (`map[mapY][mapX] == '1'`).
- Cálculo de `perpWallDist`.
- Cálculo de `lineHeight`, `drawStart` e `drawEnd`.

vii. Renderização inicial com **cores sólidas diferenciadas por orientação:**

- Norte: Vermelho (`0xFF0000`)
- Sul: Verde (`0x00FF00`)
- Leste: Azul (`0x0000FF`)
- Oeste: Amarelo (`0xFFFF00`)
- Aplicar sombreamento (`factor = 0.7`) quando `side == 1` para criar profundidade visual.

- **Como Testar:**

```
./cub3d maps/valid/simple.cub
# Você deve enxergar as paredes em 3D perfeito a partir do ponto de spawn inicial.
# Verifique que as paredes são retas (sem distorção de olho de peixe).
```

- **Git Workflow da Fase:**

```
git checkout -b feature/phase-3-raycasting-dda
git add src/raycasting/
git commit -m "feat(raycasting): motor DDA funcional com cores solidas direcionais"
git checkout develop
git merge --no-ff feature/phase-3-raycasting-dda
git push origin develop
```

Fase 4: Movimento Suave, Câmera e Colisão Básica

- **Branch:** feature/phase-4-player-movement
- **Conceitos a Estudar:**
 - Input polling desacoplado via array de teclas no loop da MLX.
 - Cinemática vetorial 2D (translação paralela e ortogonal ao vetor de direção).
 - Matriz de rotação aplicada à direção e ao plano de projeção.
- **O que Desenvolver:**
 - Sistema de estados de teclas (`t_keys` com `KeyPress` e `KeyRelease` hooks).
 - Função de atualização no `mlx_loop_hook` :
 - **W (Para frente):** avança no sentido de $\vec{dir}$ ($\Delta x = dir_x \times speed$, $\Delta y = dir_y \times speed$).
 - **S (Para trás):** recua no sentido oposto de $\vec{dir}$.
 - **A (Strafe Esquerda):** desloca no sentido perpendicular esquerdo ($\Delta x = -plane_x \times speed$, $\Delta y = -plane_y \times speed$).
 - **D (Strafe Direita):** desloca no sentido perpendicular direito ($\Delta x = plane_x \times speed$, $\Delta y = plane_y \times speed$).
 - **Setas Esquerda / Direita:** rotacionam os vetores $\vec{dir}$ e $\vec{plane}$ usando a matriz de rotação.
 - Colisão básica:** Antes de alterar a posição do jogador, verifica se a célula de destino `map[(int)(posY + deltaY)] [(int)(posX + deltaX)]` é diferente de '1' .
- **Como Testar:**

```
./cub3d maps/valid/simple.cub
# Andar com WASD e rotacionar com as setas. O movimento deve ser 100% fluido (sem travas ao segurar as tecla
# Tentar atravessar paredes: o jogador deve ser impedido de entrar na parede.
```

Git Workflow da Fase:

```
git checkout -b feature/phase-4-player-movement
git add src/player/ src/events/
git commit -m "feat(movement): input suave com WASD e rotacao da camera sem stuttering"
git checkout develop
git merge --no-ff feature/phase-4-player-movement
git push origin develop
```

Fase 5: Mapeamento de Texturas (XPM) nas Paredes

- **Branch:** feature/phase-5-textures
- **Conceitos a Estudar:**
 - Carregamento de imagens XPM com `mlx_xpm_file_to_image` .
 - Mapeamento UV e interpolação de coordenadas de textura.
 - Endereçamento de textura por fatia vertical.
- **O que Desenvolver:**
 - Carregar as 4 texturas (`NO` , `SO` , `WE` , `EA`) na inicialização do jogo para structs `t_texture` .
 - Obter ponteiro e endereço de memória de cada textura (`mlx_get_data_addr`).
 - No Raycasting:
 - Determinar qual textura usar com base em `side` , `rayDirX` e `rayDirY` .
 - Calcular `wallX` (ponto exato do impacto horizontal na parede).
 - Calcular `texX` (coluna da textura).
 - No loop vertical de desenho (de `drawStart` a `drawEnd`), calcular `texY` e transferir a cor do pixel da textura para o buffer da tela.

- **Como Testar:**

```
./cub3d maps/valid/simple.cub
# As 4 faces devem exibir suas respectivas texturas perfeitamente alinhadas.
# Checar se as texturas não estão invertidas ou borradas ao se aproximar.
```

- **Git Workflow da Fase:**

```
git checkout -b feature/phase-5-textures
git add src/render/texture_mapping.c src/init/init_textures.c
git commit -m "feat(textures): mapeamento UV completo de texturas XPM nas 4 orientacoes"
git checkout develop
git merge --no-ff feature/phase-5-textures
git push origin develop
```

Fase 6: Auditoria da Parte Obrigatória (Norma, Leaks, Edge Cases)

- **Branch:** feature/phase-6-mandatory-audit

- **Conceitos a Estudar:**

- Norminette 42 e padrões de código limpo.
- Depuração avançada com Valgrind.
- Testes de estresse com mapas anômalos.

- **O que Desenvolver:**

- i. Submeter todo o código ao norminette :
 - Máximo de 25 linhas por função.
 - Máximo de 5 variáveis por função.
 - Máximo de 4 argumentos por função.
 - Nenhuma atribuição ou operador proibido.

- ii. Bateria de testes de vazamento de memória com Valgrind:

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./cub3d maps/valid/simple.cub
```

- iii. Testar dezenas de mapas com erros intencionais para assegurar que TODOS imprimem `Error\n` com saída limpa.

- **Git Workflow da Fase:**

```
git checkout -b feature/phase-6-mandatory-audit
git add .
git commit -m "refactor(mandatory): auditoria completa de Norma e zero memory leaks"
git checkout develop
git merge --no-ff feature/phase-6-mandatory-audit
git checkout main
git merge --no-ff develop
git tag -a v1.0-mandatory -m "Parte Obrigatoria 100% Finalizada e Validada"
git push origin main develop --tags
```

Fase 7 (Bônus 1): Sistema Avançado de Colisão e Deslize

- **Branch:** feature/phase-7-bonus-collisions

- **Conceitos a Estudar:**

- Colisão por caixa delimitadora (*Bounding Box*) ou raio de colisão do jogador.
- Resolução independente de eixos (\$X\$ e \$Y\$) para permitir o efeito de deslize (*wall sliding*).

- **O que Desenvolver:**

- i. Definir um raio de colisão do jogador (ex: `COLLISION_RADIUS 0.2`).
- ii. Em vez de checar apenas a coordenada final exata, checar os 4 vértices do quadrado do jogador ao redor da nova posição:
 - `$(newX \pm radius, posY)$`
 - `$(posX, newY \pm radius)$`
- iii. **Deslizamento Suave:** Testar o movimento no eixo X e no eixo Y separadamente. Se o eixo X colidir mas o Y estiver livre, o jogador desliza pela parede em vez de travar bruscamente.

- **Git Workflow da Fase:**

```
git checkout -b feature/phase-7-bonus-collisions
git add src/player/collision.c
git commit -m "feat(bonus): colisao com bounding box e deslizamento suave em paredes"
git checkout develop
git merge --no-ff feature/phase-7-bonus-collisions
git push origin develop
```

Fase 8 (Bônus 2): Minimapa 2D Interativo no HUD

- **Branch:** `feature/phase-8-bonus-minimap`
- **Conceitos a Estudar:**
 - Composição gráfica 2D sobre o buffer 3D.
 - Desenho de formas geométricas (quadrados e linhas de Bresenham para direção).
- **O que Desenvolver:**
 - i. Definir um quadrante no canto superior da tela (ex: `$200 \times 200$` pixels).
 - ii. Desenhar a grade do mapa ao redor do jogador (ex: raio de visão de 5 blocos do mapa).
 - iii. Renderizar:
 - Paredes em cinza claro ou azul escuro.
 - Espaços vazios em preto translúcido.
 - Jogador como um ponto vermelho ou quadrado centralizado.
 - Linha indicando o vetor de visão do jogador.

- **Git Workflow da Fase:**

```
git checkout -b feature/phase-8-bonus-minimap
git add src/bonus/minimap_bonus.c
git commit -m "feat(bonus): minimapa interativo com radar do jogador no HUD"
git checkout develop
git merge --no-ff feature/phase-8-bonus-minimap
git push origin develop
```

Fase 9 (Bônus 3): Portas Interativas (Abertura/Fechamento)

- **Branch:** `feature/phase-9-bonus-doors`
- **Conceitos a Estudar:**
 - Estado dinâmico em células da grade do mapa.
 - Interação por tecla de proximidade e renderização com deslocamento.
- **O que Desenvolver:**
 - i. Adicionar o caractere `'D'` na validação do parser para identificar portas.
 - ii. Criar struct de estado da porta: `CLOSED`, `OPENING`, `OPEN`, `CLOSING`.
 - iii. No Raycasting: portas são tratadas como paredes, mas com textura específica de porta (`textures/door.xpm`).
 - iv. Interação: ao pressionar a tecla `E` ou `Espaço`, se o jogador estiver a menos de 1.5 unidades de uma porta à sua frente, altera seu estado. Portas abertas deixam o raio passar e não causam colisão física.

- **Git Workflow da Fase:**

```
git checkout -b feature/phase-9-bonus-doors
git add src/bonus/doors_bonus.c
git commit -m "feat(bonus): sistema de portas interativas abre e fecha"
git checkout develop
git merge --no-ff feature/phase-9-bonus-doors
git push origin develop
```

Fase 10 (Bônus 4): Rotação de Visão com o Rato (Mouse Look)

- **Branch:** feature/phase-10-bonus-mouse

- **Conceitos a Estudar:**

- Evento de movimento de ponteiro (`MotionNotify` no X11).
- Sensibilidade do mouse e controle contínuo de rotação.

- **O que Desenvolver:**

- Conectar o hook de evento de mouse:

```
mlx_hook(game.win, 6, 1L << 6, mouse_move_handler, &game);
```

- Calcular o deslocamento $\Delta x = x_{\text{atual}} - x_{\text{anterior}}$.

- Multiplicar Δx por uma sensibilidade (ex: 0.003) e aplicar a rotação nos vetores de direção e plano da câmera.

- Opcional/Recomendado: manter o mouse centralizado na janela (`mlx_mouse_move`) para rotação ilimitada sem sair das bordas da tela.

- **Git Workflow da Fase:**

```
git checkout -b feature/phase-10-bonus-mouse
git add src/bonus/mouse_bonus.c
git commit -m "feat(bonus): controle de visao em primeira pessoa com o mouse"
git checkout develop
git merge --no-ff feature/phase-10-bonus-mouse
git push origin develop
```

Fase 11 (Bônus 5): Sprites Animadas e Z-Buffer (Depth Buffer)

- **Branch:** feature/phase-11-bonus-sprites

- **Conceitos a Estudar:**

- Ordenação por profundidade (*Painter's Algorithm*).
- Projeção de sprites usando transformação pela matriz inversa da câmera.
- Z-Buffering para oclusão de objetos atrás de paredes.
- Transparência de cor-chave (*color keying*).

- **O que Desenvolver:**

- Alocar um array de ponto flutuante `double z_buffer[WIN_WIDTH]` que armazena a `perpwallDist` calculada para cada coluna x no raycasting das paredes.
- Carregar sprites animadas (ex: 4 frames de tocha tremeluzente `torch_1.xpm` a `torch_4.xpm`).
- No ciclo de renderização:
 - Calcular a distância de cada sprite ao jogador e ordenar os sprites do mais distante para o mais próximo.
 - Transformar a coordenada do sprite para o espaço de câmera do jogador:
$$\begin{pmatrix} \text{transfX} \\ \text{transfY} \end{pmatrix} = \frac{1}{\det} \begin{pmatrix} \text{dirY} & -\text{dirX} \\ -\text{planeY} & \text{planeX} \end{pmatrix} \begin{pmatrix} \text{spriteX} - \text{posX} \\ \text{spriteY} - \text{posY} \end{pmatrix}$$

- Calcular a posição e altura do sprite na tela.
- Desenhar apenas se o pixel do sprite não for transparente (ignorar cor-chave preta `0x000000` ou magenta `0xFF00FF`) e se $\text{transfY} < z_buffer[\text{stripeX}]$ (o sprite está à frente da parede daquela coluna).

- **Git Workflow da Fase:**

```
git checkout -b feature/phase-11-bonus-sprites
git add src/bonus/sprites_bonus.c
git commit -m "feat(bonus): sprites animados com z-buffer e oclusao de paredes"
git checkout develop
git merge --no-ff feature/phase-11-bonus-sprites
git push origin develop
```

Fase 12: Integração Final, README Oficial e Preparação para Defesa

- **Branch:** feature/phase-12-final-delivery
- **Conceitos a Estudar:**
 - Diretrizes formais do Chapter VI e IX do subject.
 - Verificação de código por pares e simulação de arguição.
- **O que Desenvolver:**
 - i. Criação do `README.md` rigorosamente formatado de acordo com o subject:
 - Linha 1 em itálico: *Este projeto foi criado como parte do currículo da 42 por hcosta, hcuessec*
 - Seção **Descrição** completa.
 - Seção **Instruções** de compilação (`make`, `make bonus`) e execução com exemplos de mapas.
 - Seção **Recursos** com referências técnicas e a declaração explícita do uso ético de IA (explicando para quais tarefas de design e arquitetura foi utilizada).
 - ii. Regra `bonus` do Makefile gerando o executável com todos os recursos extras ativos sem erros de compilação ou warnings.
 - iii. Bateria de testes finais de estresse:
 - 100% de conformidade com a `norminette`.
 - 0 vazamentos de memória no Valgrind em todos os caminhos.
 - Testes de alteração rápida de código durante a defesa (ex: mudar velocidade do jogador, trocar textura ou alterar cor do chão dinamicamente).
- **Git Workflow da Fase:**

```
git checkout -b feature/phase-12-final-delivery
git add README.md Makefile src/
git commit -m "docs: readme oficial e preparativos finais para avaliacao"
git checkout develop
git merge --no-ff feature/phase-12-final-delivery
git checkout main
git merge --no-ff develop
git tag -a v2.0-bonus -m "Versao Final Completa com Todos os Bonus"
git push origin main develop --tags
```

6. Manual de Assets: Criação, Obtenção e Conversão para XPM

6.1 Onde Obter Assets Gratuitos e Legais

Para o cub3D ter um visual profissional estilo *retro-FPS*, você deve utilizar texturas com licença livre (CC0 / Domínio Público / MIT):

1. **Kenney.nl:** Procure pelo pacote "*Retro Medieval Kit*" ou "*Dungeon Kit*" (contém texturas de tijolos de pedra, portas de madeira e grades medievais de altíssima qualidade).
2. **OpenGameArt.org:** Busque por termos como "*Wolfenstein texture pack*", "*dungeon walls 64x64*", "*pixel art torch*".
3. **itch.io:** Seção *Game Assets* filtrada por *Pixel Art* e *2D/Textures*.

6.2 Ferramentas Recomendadas para Criação e Edição

- **Aseprite / LibreSprite:** O padrão absoluto para desenhar pixel art e animações quadro a quadro (tochas, moedas, monstros).
- **GIMP:** Excelente para redimensionar, ajustar paletas e exportar diretamente para `.xpm`.

6.3 Especificações Técnicas Fundamentais

Para garantir performance máxima e facilidade matemática de cálculo:

- **Resolução Ideal:** 64×64 pixels por textura (ou 128×128).
- **Por que potências de 2 (2^n)?**
Permite otimização extrema no cálculo do índice do pixel. Em vez de usar a divisão e o operador de módulo (`texX = (int)(wallX * 64) % 64`), pode-se usar uma máscara binária bitwise rápida (`texX & 63`).
- **Paleta de Cores Coerente:** Limitar-se a uma paleta de 16 a 32 cores cria uma atmosfera retrô autêntica e harmônica.

6.4 Conversão de PNG / JPG para XPM no Linux

A miniLibX suporta nativamente o formato **XPM (X PixMap)** através da função `mlx_xpm_file_to_image`.
Se você baixar arquivos em `.png`, converta-os facilmente no terminal usando a ferramenta `ImageMagick`:

```
# Instalar ImageMagick se necessário
sudo apt-get install imagemagick

# Converter imagem PNG de 64x64 para XPM com profundidade de 8 bits
convert texture_north.png -resize 64x64! -depth 8 textures/wall_north.xpm
convert texture_south.png -resize 64x64! -depth 8 textures/wall_south.xpm
convert texture_west.png -resize 64x64! -depth 8 textures/wall_west.xpm
convert texture_east.png -resize 64x64! -depth 8 textures/wall_east.xpm

# Converter frames de animação para o bônus de sprites
convert torch_anim_01.png -resize 64x64! -depth 8 textures/sprites/torch_01.xpm
convert torch_anim_02.png -resize 64x64! -depth 8 textures/sprites/torch_02.xpm
```

Tratamento de Transparência para Sprites:

Nos arquivos `.xpm` de sprites, verifique se a cor de fundo transparente está definida como `"None"` ou preencha o fundo com uma cor mágica (como Magenta `0xFF00FF`). No código de renderização do sprite, simplesmente adicione a condição:

```
if ((color & 0x00FFFFFF) == 0x00FF00FF || color == 0x000000)
    continue ; // Ignora o pixel transparente, revelando a parede ao fundo
```

7. Estimativa de Prazos e Cronograma a partir de Hoje

Data de Início: 19 de Setembro de 2026

Carga Horária Estimada: 280 horas de trabalho no total (140 horas por integrante em dupla).

7.1 Cenário A: Trabalho em Dupla / Pair Programming (Recomendado)

Dedicação regular de 20 a 25 horas semanais por pessoa (cerca de 3 a 4 horas por dia útil).

Semana / Período	Fases de Trabalho	Entregas e Validações	Status Esperado
Semana 1 (19/09 – 26/09)	Fase 0 & Fase 1 (Setup, Libft, Parsing Inicial)	Makefiles, structs partilhadas, leitura de caminhos e cores.	Janela abre, arquivo .cub é lido.
Semana 2 (27/09 – 04/10)	Fase 1 (Parsing de Mapa e Flood Fill)	Validação geométrica completa de labirintos válidos e inválidos.	Rejeita mapas abertos com <code>Error\n</code> .
Semana 3 (05/10 – 12/10)	Fase 2 & Fase 3 (Framebuffer & DDA Raycasting)	Render de teto/chão, cálculo DDA e paredes em cores sólidas.	Mundo 3D navegável em cores puras.
Semana 4 (13/10 – 20/10)	Fase 4 (Movimento WASD e Câmera)	Input polling sem atrasos, rotação vetorial e colisões básicas.	Jogador anda e vira suavemente.
Semana 5 (21/10 – 28/10)	Fase 5 (Texturização XPM)	Mapeamento UV de texturas nas paredes Norte, Sul, Leste e Oeste.	Visual completo do jogo obrigatório.
Semana 6 (29/10 – 05/11)	Fase 6 (Auditoria Obrigatória e Refatoração)	100% Norminette, zero leaks com Valgrind, freeze da parte obrigatória.	PARTE OBRIGATÓRIA CONCLUÍDA 🏆
Semana 7 (06/11 – 13/11)	Fase 7 & Fase 8 (Colisões Avançadas e Minimapa)	Deslizamento em paredes e minimapa dinâmico no HUD.	Bônus 1 e 2 funcionais.
Semana 8 (14/11 – 21/11)	Fase 9, 10 & 11 (Portas, Mouse Look, Sprites)	Interação com portas, rotação com mouse e animações de tocha.	Todos os 5 bônus implementados.
Semana 9 (22/11 – 27/11)	Fase 12 (README, Peer Review e Defesa)	README oficial, testes cruzados com outros estudantes da 42.	Pronto para Submissão e 125%!

Previsão de Conclusão em Dupla: Final de Novembro de 2026 (aproximadamente 65 a 70 dias corridos).

7.2 Cenários Alternativos de Ritmo

- Modo Intensivo / Imersão (40h/semana por pessoa em tempo integral):**
Pode ser concluído em aproximadamente **30 a 35 dias corridos** (~25 de Outubro de 2026).
- Modo Individual ou Part-Time Reduzido (10h a 12h semanais):**
Estimativa de **10 a 12 semanas** (~final de Dezembro de 2026).

8. Erros Clássicos a Evitar e Dicas para a Avaliação por Pares

8.1 As Armadilhas Mais Comuns que Reprovam no cub3D

- Divisão por Zero no DDA:**
Se o raio for perfeitamente paralelo ao eixo (`$rayDirX == 0$` ou `$rayDirY == 0$`), calcular `1.0 / rayDirX` causará `inf` ou divisão por zero. Sempre trate isso atribuindo um valor muito alto (ex: `1e30`) para a distância delta.
- Vazamento de Memória ao Sair com Erro:**
Se o parser falhar na linha 40 do mapa, muitos esquecem de liberar as texturas e matrizes já alocadas nas linhas anteriores. Crie uma rotina centralizada `clean_exit(game)` que passe por todos os ponteiros não-nulos e dê `free()` /

```
mlx_destroy_image() .
```

3. Destruir o Display Antes das Imagens:

No Linux com a MiniLibX, SEMPRE destrua todas as imagens criadas (`mlx_destroy_image`) ANTES de fechar a janela (`mlx_destroy_window`), fechar o display (`mlx_destroy_display`) e liberar o ponteiro da MLX (`free(mlx_ptr)`). Inverter a ordem causa *Invalid Read/Write* no Valgrind.

4. Espaços Internos no Mapa:

Um espaço ' ' cercado por paredes '1' é válido segundo o subject. O que NÃO pode ocorrer é um espaço vazio '0' encostar em um espaço ' ' ou na borda do mapa.

5. Re-link no Makefile:

Verifique se o seu Makefile não recompila objetos se nada foi alterado. Rode `make` , depois `make` novamente. A segunda chamada deve responder `make: Nothing to be done for 'all'` .

8.2 Como Brilhar na Avaliação por Pares

Conforme descrito no **Capítulo IX do subject**, o avaliador pode pedir que você faça uma **pequena alteração de código ao vivo** em poucos minutos (por exemplo: inverter a velocidade do jogador, alterar uma cor ou modificar um parâmetro de raio).

- **Ambos os parceiros devem conhecer 100% do código:** Mesmo que um tenha focado no parsing e o outro no motor de raycasting, façam sessões de *Code Walkthrough* onde cada um explica detalhadamente suas funções para o outro.
- **Saiba explicar a matemática do DDA e a eliminação do Fisheye:** Nenhum avaliador aceita "peguei da internet e funcionou". Desenhe num papel o triângulo retângulo, mostre a projeção no plano da câmera e explique por que a distância perpendicular resolve a distorção.

Com este roteiro em mãos, seu caminho para um cub3D impecável com 125% está completamente traçado. Bom desenvolvimento!